Algorithmic Differentiation: A Practical Approach for Engineers

Short Course (Day 1)

Pavanakumar Mohanamuraly

Excerpts from "Numerical Optimisation" course by Dr. Jens D Mueller, Queen Mary University of London. *(notes and figures with permission)*

ॐ पूर्णमदः पूर्णमिदं पूर्णात्पूर्णमुदच्यते ।

पूर्णस्य पूर्णमादाय पूर्णमेवावशिष्यते ॥

Axiom (Additive Identity)

For any $x \in \mathbb{R}$ $(x \neq 0)$:

1. $x + 0 = x$
2. $x - 0 = x$
3. $0 + 0 = 0$
4. $0 - 0 = 0$

ॐ शान्तिः शान्तिः शान्तिः ॥

Tools used in the course

Python + PyTorch

Automatic Differentiation via `torch.autograd`

Github: `https://github.com/pavanakumar/IIT_ISM`

Exercises: `exercises/day1`

```
git clone https://github.com/pavanakumar/IIT_ISM.git
```

Motivation

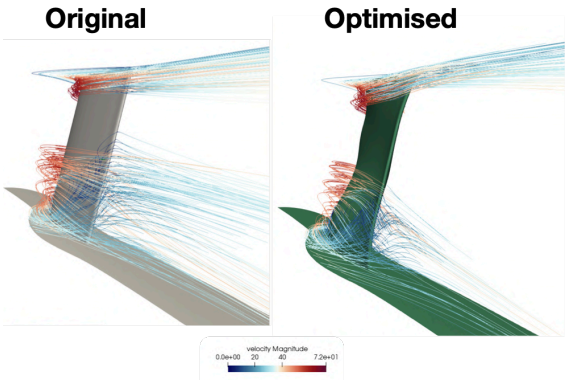
Course deals with mathematical derivatives & applications to optimisation and machine learning

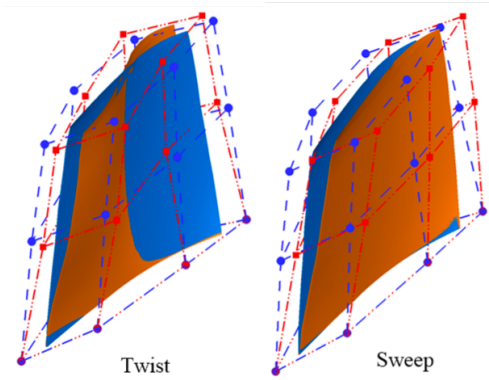
$$\frac{dy}{dx} \quad \nabla f$$

Why do we need derivatives?

Why are they important?

Aerodynamic Shape Optimisation – Streamlines

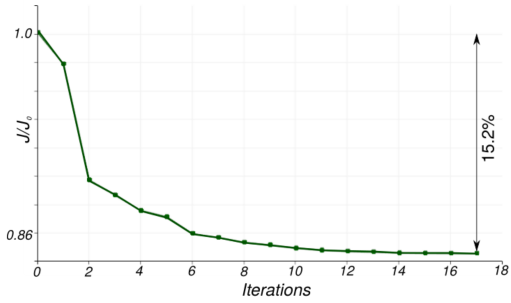


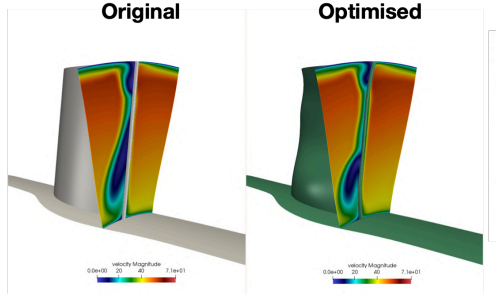


192 blade parameters (parametric CAD), 4 constraints

Source: O Mykhaskiv et al, CAD 15(6), 2018.

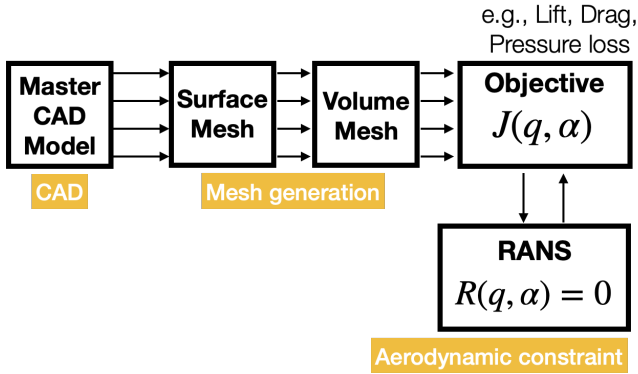
Aerodynamic Shape Optimisation – Results





Result: 15% lower entropy loss in 17 iterations (mass-flow and geometric constraints satisfied)

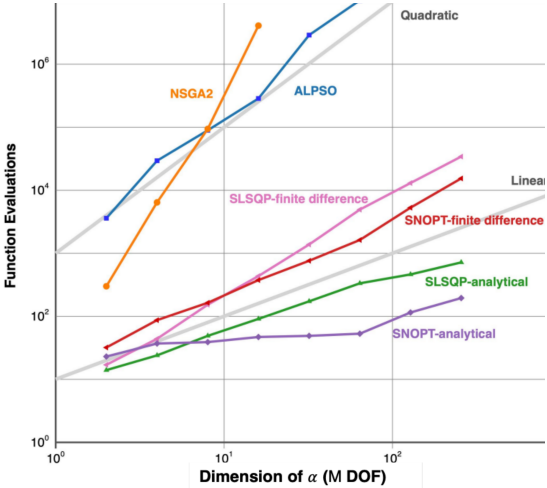
Optimization Pipeline



$q(\alpha) \Rightarrow$ State (N degrees of freedom)
$\alpha \Rightarrow$ Design (M parameters)

Goal: Compute $\frac{dJ}{d\alpha}$ efficiently

Gradient-based vs Gradient-free Methods



Gradient-based (linear complexity)

- Well suited to large # DOF
- Requires derivative $\frac{dJ}{d\alpha}$
- SLSQP, SNOPT with analytical gradients

Gradient-free (quadratic complexity)

- Suitable for smaller # DOF
- Works with black-box models
- NSGA2, ALPSO

* Lyu, et al. Benchmarking optimization algorithms for wing aerodynamic design optimization. ICCFD8-2014.

Rosenbrock Function

minimize

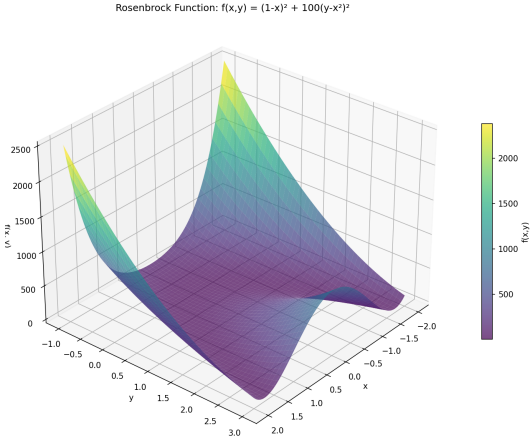
$$\sum_{i=1}^{n-1} 100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2$$

$\boldsymbol{N = 2}$: Global min at $(1, 1)$, $f = 0$
$\boldsymbol{N = 3}$: Global min at $(1, 1, 1)$
$\boldsymbol{N \geq 7}$: No analytical solution

Constrained Benchmark

$$\textbf{s.t.} \sum_{i=1}^{n-1} \left(1.1 - (x_i - 2)^3 - x_{i+1}\right) \geq 0$$

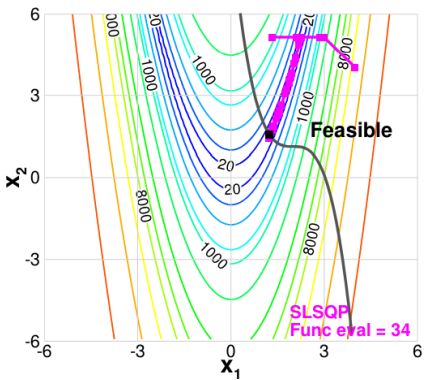
$\boldsymbol{N = 2}$: Global min at $(1.24, 1.54)$
x, f tolerance $= 1.0e - 6$



Source: Wikipedia

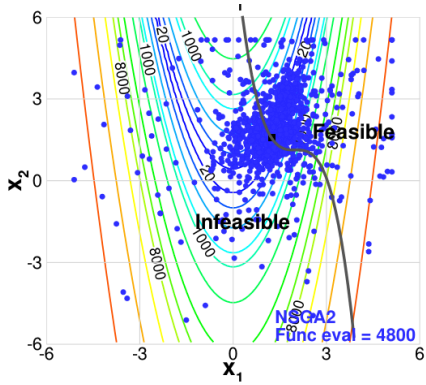
Gradient-based vs Gradient-free Optimization

SLSQP (Gradient-based)



34 evaluations

NSGA2 (Gradient-free)



4800 evaluations

Gradient information enables **140x faster** convergence!

Deep Learning: The Basic Building Block

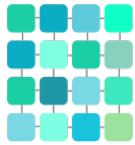
$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{z} = \sigma(\mathbf{y})$$

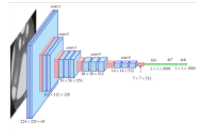
- Simple function approximation with tuneable parameters
- Deep combinatorics produces rich representations
- Parameters optimised using **gradient-based** methods

TEMPORAL/ SEQUENCE

Recurrent neural network (RNN)

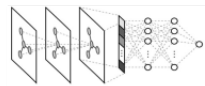


REGULAR TOPOLOGY



Convolutional neural network (CNN)

RELATIONAL DATA AND UNSTRCUTRED TOPOLOGY



Graph network (GNN/GCN)

Deep Learning and Derivatives

Linear Kernel Activation

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad \longrightarrow \quad \mathbf{z} = \sigma(\mathbf{y})$$

Find $\mathbf{w} = [\mathbf{W}\ \mathbf{b}]$ that minimise $J(\mathbf{w})$ given data (batch size n) $J_i(\mathbf{w})$

Stochastic gradient descent (η is the step size)

$$\mathbf{w}^{new} = \mathbf{w}^{old} + \frac{\eta}{n} \sum_{i=1}^{n} \nabla J_i(\mathbf{w})$$

Algorithmic Differentiation and gradient-based optimization are behind virtually all recent successes in machine learning

Computing Derivatives

How do we obtain these gradients?

We've seen WHY derivatives matter...

Now let's explore HOW to compute them

Approaches to derivative calculation

- *Symbolic Differentiation*
  - Manually derive and code into program
  - Software based with code generation (Maxima)
- *Approximate Numerical Differentiation*
  - Forward or central difference
- *Complex-step method and (Hyper)Dual numbers*
- *Algorithmic Differentiation*
  - Overloading
  - Source Transformation

What isn't AD? – Symbolic Differentiation

Automatic manipulation of expressions using differentiation rules:

$$\frac{d}{dx}(f+g) \rightsquigarrow \frac{df}{dx} + \frac{dg}{dx} \qquad \frac{d}{dx}(fg) \rightsquigarrow f'g + fg'$$

- Mechanical process of differentiating expression trees
- Automation via computer algebra systems (Maxima, SymPy, etc.)
- Provides analytical insight into problem structure
- **Drawback:** Expression blowup – derivatives become exponentially larger
- **Drawback:** Numerical stability issues

What isn't Algorithmic Differentiation?

Finite-difference

Approximate the derivative using truncated Taylor series expansion. Simplest version is the forward difference,

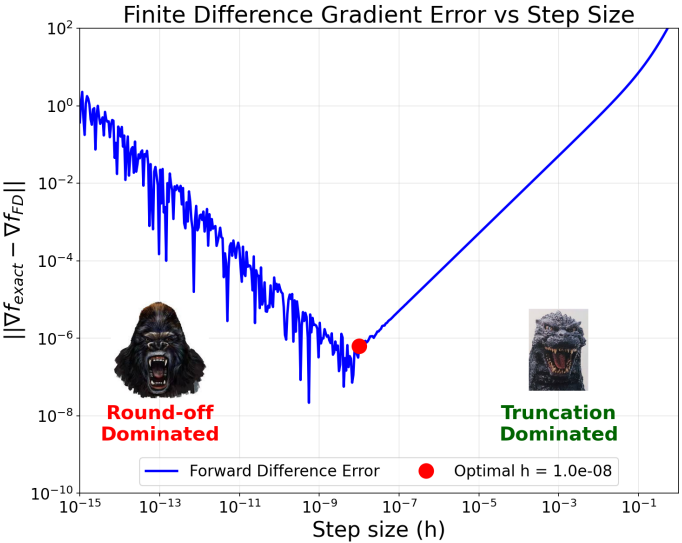
$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h} + O(h)$$

with h a small perturbation size and $\mathbf{e}_i$ a vector of the same length as x with zeros everywhere, but one in position i. A better approximation is the central difference,

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h} + O(h^2)$$

Can we let $h \to 0$ to make the error vanish?

Truncation vs. Round-off Error



- Optimal h where both errors minimal
- Problem/scale dependent
- Standard for black-box models
- Useful for gradient verification

Finite-difference for gradient computation

- Needs carefully setting optimal choice for step-size h
  - *Too small $h \rightarrow$ large round-off (cancellation)*
  - *Too large $h \rightarrow$ large truncation error*

- Forward difference: T.E. $\propto \mathcal{O}(h)$ *with one additional evaluation per parameter (or design variable)*

- Central difference: T.E. $\propto \mathcal{O}(h^2)$ *with two additional evaluation per parameter (or design variable)*

- Usually for n design variables require $n + 1$ function evaluations (for gradient) !

Derivatives using Complex-step method

Consider the Taylor series expansion of the real valued function $f(x)$ with step-size h in the complex plane $F(x+ih)$

$$F(x+ih) = F(x) + ihF'(x) - \frac{h^2}{2!}F'(x) - \frac{ih^3}{3!}F''(x) + \mathcal{O}(h^4)$$

Taking the imaginary part and dividing by h we obtain,

$$\frac{1}{h}\text{Im}\Big(F(x+ih)\Big) = F'(x) - \frac{h^3}{3!}F'''(x) + \mathcal{O}(h^4)$$

$$\boxed{F'(x) = \frac{1}{h}\text{Im}\Big(F(x+ih)\Big) + \mathcal{O}(h^2)}$$

The complex variable "trick"

Properties of complex-step gradient computation

$$F'(x) = \frac{1}{h}\mathrm{Im}\Big(F(x+ih)\Big) + \mathcal{O}(h^2)$$

- No cancellation error problem, so choose h to be arbitrarily small to make T.E. as small as one wishes
- Approximation error: T.E. $\propto \mathcal{O}(h^2)$ *same as central difference*
- For n design variables require n function evaluations (linear)
- Cost of complex arithmetic is higher than real-valued computations (SIMD, Compiler/Hardware optimisations, etc.)
- Can be implemented easily in languages that have support for complex arithmetic
  - Redefine non-analytic functions such as *abs*, $<$, $>$, etc.

Algorithmic Differentiation

The Best of Both Worlds

- Exact (not approximate)
- Efficient (not $\mathcal{O}(n)$ cost)
- Stable (no truncation/roundoff issues)
- Automated (no expression blowup)

What is Algorithmic Differentiation?

- Program computes $f(x)$ as sequence of elementary operations ($+$, $-$, $\times$, $\div$, $\sin$, $\cos$, ...)

$$f(\mathbf{x}) = f_n\Big(f_{n-1}\big(\cdots f_2(f_1(\mathbf{x}))\big)\Big)$$

- Chain-rule gives derivative (forward mode):

$$\frac{\partial f}{\partial x_i}\dot{x} = \frac{\partial f_n}{\partial f_{n-1}} \cdot \frac{\partial f_{n-1}}{\partial f_{n-2}} \cdots \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial x_i}\dot{x} = E_n E_{n-1} \cdots E_1 \dot{x}$$

- Process can be **automated** – hence *Automatic Differentiation*

Forward mode using an example

$$f(\mathbf{x}) = 3x_1 + 2x_2 + x_3, \quad \mathbf{x} = \{x_1, x_2, x_3\}$$

$$\frac{\partial f}{\partial x_1} = 3, \ \frac{\partial f}{\partial x_2} = 2, \ \frac{\partial f}{\partial x_3} = 1$$

- Forward mode AD gives directional gradient
- Input $\mathbf{x}_d \rightarrow$ output $u_d = \nabla f \cdot \mathbf{x}_d$
- $\mathbf{x}_d$ is the **seed vector**
- Seeds $\{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\}$ give full Jacobian

```
def f(x):
    return 3*x[0] + 2*x[1] + x[2]
```

```
def f_d(x, xd):
    u  = 3*x[0] + 2*x[1] + x[2]
    ud = 3*xd[0] + 2*xd[1] + xd[2]
    return u, ud
```

Reverse mode AD

- Recall that the forward mode AD computes gradient of $y = f(x)$

$$\dot{y} = \frac{\partial f}{\partial x}\dot{x} = E_n E_{n-1} \cdots E_2 E_1 \dot{x}$$

- Applying a simple transpose rule of matrix multiplication to $\bar{y}\frac{\partial f}{\partial x}$ we get:

$$\left(\bar{y}\frac{\partial f}{\partial x}\right)^T = E_1^T E_2^T \cdots E_{n-1}^T E_n^T \bar{y}^T$$

- We apply the same differentiation operator E_i as forward mode but use their transpose (apply chain rule and accumulate starting from E_n)
- We follow the primal statement in reverse
- Another convention to represent reverse mode (transpose dropped):

$$\bar{x} = \bar{x} + \left(\frac{\partial f}{\partial x}\right)^T \bar{y}$$

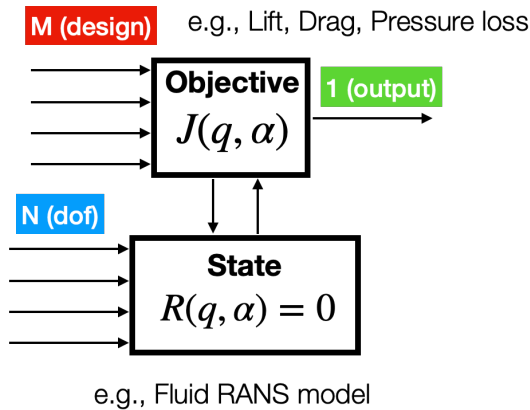
Adjoint Primer: Constraint Optimisation

Constraint Optimisation

$$\min_{\alpha} J(q, \alpha)$$

s.t. $R(q, \alpha) = 0$

- q – State (N degrees of freedom)
- α – Design (M parameters)
- J – Objective (scalar output)
- R – State equation (constraint)



Adjoint Primer: A Simple Example (Linear Objective and State)

State equation:

$$R(q, \alpha): \quad \mathbf{A}q = b \quad (\text{or } \mathbf{A}q - b = 0)$$

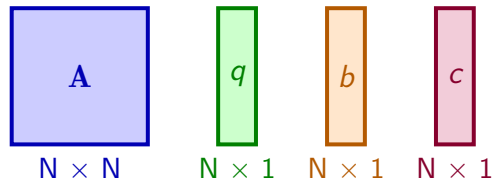
Objective function:

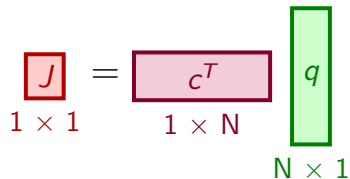
$$J(q, \alpha) \equiv c^T q$$

Note:

- q – $q(\alpha)$, state vector
- b – $b(\alpha)$, forcing term
- c – constant vector

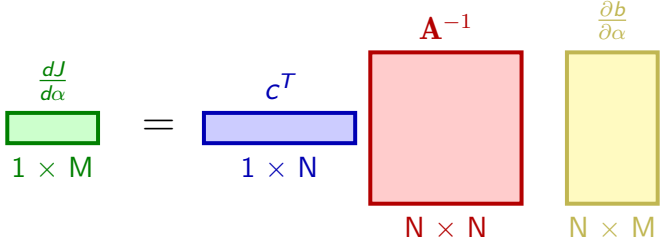
Matrix/Vector Shapes





Adjoint Primer: Direct Sensitivity (Forward)

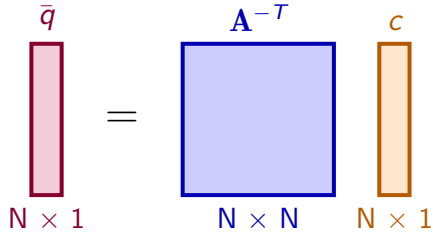
$$\frac{dJ}{d\alpha} = \cancel{\frac{\partial J}{\partial \alpha}} + \frac{\partial J}{\partial q}\frac{\partial q}{\partial \alpha} = c^T \mathbf{A}^{-1} \frac{\partial b}{\partial \alpha}$$



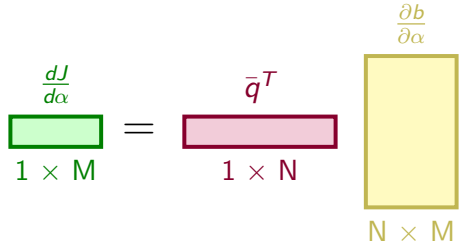
Cost: O(M) linear solves (One solve per design variable)

Adjoint Primer: Adjoint Sensitivity (Reverse)

$\mathbf{A}^T \bar{q} = c$:

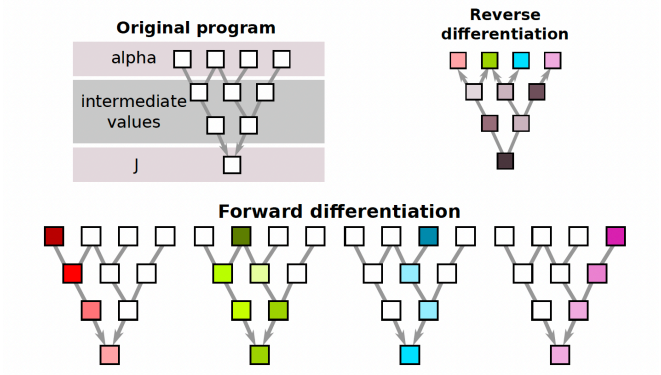


$\frac{dJ}{d\alpha} = \bar{q}^T \frac{\partial b}{\partial \alpha}$:



Cost: O(1) – Solve adjoint once, then matrix-vector for any M!

Forward vs Reverse Mode: Visual Comparison



Forward: Input $\rightarrow$ Output

One reverse pass gives ALL derivatives!

Reverse: Output $\rightarrow$ All Inputs

Source: Jens Mueller, QMUL

From Algorithmic to Automatic Differentiation

- Forward-mode steps through the statements in the same order, add a derivative computation statement before each primal statement.
- The reverse-mode records all statements, then accumulates their derivatives in reverse.
- Both are straightforward (i.e. rigorous, rule-based, mechanical) process why not have this done by software.

There are two main ways to apply automatic differentiation

- *Source-transformation AD*
- *Operator-overloading AD*

Summary

- Analytic derivatives are limited to small problems (expression blowup)
- Finite-differences are widely used, simple to implement, but have linear cost and require careful choice of step size
- Complex-step can be implemented effectively in some languages, if the source code is available. No truncation error, but linear cost
- Algorithmic differentiation (AD) needs application at source code level, are exact, cost is linear in forward mode
- AD can be extended to the reverse mode, which is exact and has constant cost

अन्धं तमः प्र विशन्ति येऽविद्यामुपासते ।

ततो भूय इव ते तमो य उ विद्यायाꣳ रताः ॥९॥

Those who worship what is not real knowledge (avidya), enter into blind darkness; those who delight in real knowledge (vidya), enter, as it were, into greater darkness.